

FALCON-1 PoC

기술 분석 보고서 (Tech Analysis Report)

영역 I — LLM · Reasoning (6종)

[accent : SLATE]

프로젝트 : FALCON-1 (Flow-Assisted Logistics & Collaborative Operation Network — One)

팀 : NERDLABS | 버전 : v0.1 | 작성일 : 2026-04-30

작성자 : 이건희 (NERDLABS 팀장) | 검토 : CTO 페르소나

보고서 BLUF

BLUF FALCON-1 LLM-Reasoning 영역 6종 분석 결과: Function calling layer (Pydantic) + llama.cpp/Ollama runtime = MUST 2종 (LLM 도입 시 hallucination 방지 + 실행 인프라), Gemma 2 2B 또는 Qwen 2.5 1.5B = SHOULD (한국어 강함, BRAVO 8GB 가능), Llama 3.2 1B + Phi-3.5 mini = 대안. v1.0 PoC는 LLM 통합이 critical path 아님 — Sprint 4 시간 여유 시 제한적 명령 어휘 (10-15개 동사·5종 명사) → JSON action 변환에만 활용. 한국어 명령 95%+ 처리는 Function calling layer가 핵심, LLM 자체보다 Pydantic schema 검증이 안정성 결정.

본 보고서는 영역 G(faster-whisper 한국어 ASR)을 전제로 한다. ASR 출력 한국어 텍스트가 본 영역 LLM pipeline으로 들어가 JSON action으로 변환되고, 그 JSON이 영역 D(Movelt) + C(Nav2) action server로 dispatch된다.

본 영역의 accent 색상은 SLATE BLUE (사고·추론 의미)을 사용한다.

영역 I — 6종 요약 매트릭스

#	기술	역할	한국어	VRAM	권장도	적용
I1	Function calling layer	Pydantic schema 검증 + dispatch	—	—	★★★★★	MUST
I2	llama.cpp / Ollama	LLM runtime (GGUF Q4)	—	—	★★★★★	MUST
I3	Gemma 2 2B	Google, 한국어 강함	✓강함	1.5 GB Q4	★★★★★	SHOULD primary
I4	Qwen 2.5 1.5B/3B	Alibaba, 한국어 우수	✓우수	1.0 / 2.0 GB Q4	★★★★★	SHOULD 대안
I5	Llama 3.2 1B/3B	Meta, 한국어 보통	□ 보통	0.7 / 2.0 GB Q4	★★★	대안 (RPi5 fallback)
I6	Phi-3.5 mini	Microsoft 3.8B	□ 보통	2.5 GB Q4	★★★	대안

11. Function calling layer (Pydantic schema)

BLUF ★★★★★ MUST 채택. LLM 도입 결정 시점부터 가장 critical한 layer — LLM 자유 텍스트 출력을 strict JSON schema로 강제 검증하여 hallucination 차단. Pydantic v2 model로 action 정의, instructor 또는 outlines 라이브러리 활용. v1.0 PoC는 한정 어휘 (10-15 동사) 직접 매핑이 LLM 호출보다 reliable + 빠름, LLM은 모호 명령에만 fallback.

1. 개요

항목	내용
역할	LLM 출력 → strict JSON action 변환 + 검증
라이선스	Pydantic MIT, instructor BSD, outlines Apache 2.0
핵심 라이브러리	Pydantic v2 + instructor (또는 outlines)
검증 방식	Pydantic model + JSON schema + retry
대상 LLM	Gemma 2 / Qwen 2.5 / Llama 3.2 (function calling 지원)

2. 적용 방법

2.1 Pydantic Action Schema 정의 (FALCON-1)

```
# falcon1_actions.py
from pydantic import BaseModel, Field
from typing import Literal, Optional, Union
from enum import Enum

class ToolType(str, Enum):
    DRIVER = 'driver'
    HAMMER = 'hammer'
    WRENCH_14MM = 'wrench_14mm'

class BoxColor(str, Enum):
    RED = 'red'
    BLUE = 'blue'
    GREEN = 'green'
```

```

# Action 1: 공구 분류 (Pick-and-Place)
class SortToolsAction(BaseModel):
    """공구를 색상별 box로 분류"""
    action: Literal['sort_tools'] = 'sort_tools'
    tools: list[ToolType] = Field(default_factory=list,
        description='분류할 공구 목록 (빈 리스트 = 모두)')
    speed: Literal['slow', 'normal', 'fast'] = 'normal'

# Action 2: 공구 가져오기 (Handover)
class FetchToolAction(BaseModel):
    """특정 공구를 사람에게 전달"""
    action: Literal['fetch_tool'] = 'fetch_tool'
    tool: ToolType
    target: Literal['user', 'workzone'] = 'user'

# Action 3: 도킹
class DockAction(BaseModel):
    """도킹 zone으로 이동 + 충전"""
    action: Literal['dock'] = 'dock'

# Action 4: 정지
class StopAction(BaseModel):
    """안전 정지"""
    action: Literal['stop'] = 'stop'
    reason: Optional[str] = None

# Union (LLM 출력 후 자동 분기)
Action = Union[SortToolsAction, FetchToolAction,
    DockAction, StopAction]

```

2.2 LLM 호출 (instructor 라이브러리)

```

# falcon1_llm_node.py
import instructor
from openai import OpenAI
from falcon1_actions import Action, FetchToolAction

# Ollama OpenAI-compatible API 사용
client = instructor.from_openai(
    OpenAI(base_url='http://localhost:11434/v1',
        api_key='ollama'),
    mode=instructor.Mode.JSON)

system_prompt = '''당신은 FALCON-1 협업 로봇입니다.
사용자의 한국어 명령을 정확한 JSON action으로 변환하세요.
공구: driver(드라이버), hammer(망치), wrench_14mm(스패너 14mm)
Box: red(빨강), blue(파랑), green(초록)
행동: sort_tools(공구 분류), fetch_tool(가져오기),
    dock(도킹), stop(정지)
...

```

```
def korean_to_action(text: str) -> Action:
    """한국어 명령 → Action 객체 (검증 자동)"""
    response = client.chat.completions.create(
        model='gemma2:2b-instruct-q4_K_M',
        response_model=Action,
        messages=[
            {'role': 'system', 'content': system_prompt},
            {'role': 'user', 'content': text},
        ],
        max_retries=2, # 검증 실패 시 자동 재시도
    )
    return response

# 사용 예
action = korean_to_action("드라이버 가져다 줘")
# → FetchToolAction(action='fetch_tool', tool='driver', target='user')
```

2.3 v1.0 권장 아키텍처 — Direct Mapping First, LLM Fallback

```
# v1.0 권장: Rule-based primary + LLM fallback
import re

DIRECT_PATTERNS = {
    r'(드라이버|driver).*(가져|줘|건네)':
        FetchToolAction(tool='driver'),
    r'(망치|hammer).*(가져|줘|건네)':
        FetchToolAction(tool='hammer'),
    r'(스패너|wrench).*(가져|줘|건네)':
        FetchToolAction(tool='wrench_14mm'),
    r'(공구|도구).*(분류|정리)':
        SortToolsAction(),
    r'(도킹|충전|쉬어)':
        DockAction(),
    r'(정지|멈춰|stop)':
        StopAction(reason='user_command'),
}

def parse_command(text: str) -> Action:
    # Step 1: Rule-based (빠름, 결정적)
    for pattern, default_action in DIRECT_PATTERNS.items():
        if re.search(pattern, text):
            return default_action

    # Step 2: LLM fallback (모호 명령)
    try:
        return korean_to_action(text)
    except Exception as e:
        return StopAction(reason=f'parse_failed: {e}')
```

3. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
CPU (Pydantic)	<1%	✓ RPi5
LLM runtime (I2)	GPU 또는 강한 CPU	✓ BRAVO 또는 RPi5 (Q4)
RAM	100 MB (instructor)	✓
VRAM	I2 LLM 의존	✓ BRAVO 8GB 충분
latency	Rule-based <5ms, LLM 200-500ms	✓ Rule-based primary

4. 장점

1. Hallucination 차단 — Pydantic schema 검증으로 잘못된 action 자동 거부
2. Type safety — IDE 자동완성 + mypy 검증
3. Rule-based primary — 한정 어휘는 LLM 우회로 빠름·결정적
4. LLM fallback — 모호·새로운 명령에만 LLM 호출, 비용 절감
5. instructor 라이브러리 — 자동 retry · validation 통합
6. Function calling 지원 LLM — Gemma 2, Qwen 2.5, Llama 3.2 모두 OK
7. Pydantic v2 ~10x 빠름 — Rust core
8. ROS 2 통합 — JSON action → ROS 2 service/action call 표준 매핑

5. 단점·한계

9. 초기 schema 설계 부담 — v1.0 4-5 action 정의 + 제약 조건
10. Rule-based regex 한계 — 한국어 다양 표현 누락 가능
11. LLM fallback 복잡성 — instructor 학습곡선
12. Schema 변경 시 LLM 재학습 불필요하나 prompt 갱신 의무
13. Edge case — '망치 들어' vs '망치 가져' 같은 미묘 차이 처리

14. Multi-step action 처리 — 'A 가져온 후 B' 같은 복합 명령 (v1.1+)

6. 대안 비교

항목	Pydantic + instructor	OpenAI Function Calling API	JSON mode (raw)	Direct regex only
라이선스	MIT + BSD	API 비용	—	—
오프라인	✓	✗	✓	✓
검증	✓자동 retry	✓	수동	—
복잡 명령	✓LLM	✓LLM	약함	✗
latency	200-500ms	300-800ms (cloud)	200-500ms	<5ms
FALCON-1	✓MUST	비채택 (외부)	비권장	primary 보조

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST 채택 (LLM 도입 시 의무)
도입 시점	Sprint 4 (Integrated Demo)
처리 위치	RPi5 (Pydantic) + BRAVO (LLM I2)
Action 정의 (v1.0)	4-5개: SortTools, FetchTool, Dock, Stop, (선택) Speak
primary 전략	Rule-based regex (빠름, 결정적)
fallback 전략	LLM via instructor (모호 명령)
검증	Pydantic v2 schema + max_retries=2
주요 의존성	I2 llama.cpp/Ollama, I3 Gemma 2 또는 I4 Qwen 2.5, G1 faster-whisper

8. 참고

- Pydantic v2 : <https://docs.pydantic.dev/>
- instructor : <https://python.useinstructor.com/>
- outlines (대안) : <https://github.com/dottxt-ai/outlines>
- Function calling 가이드 : <https://ollama.com/blog/tool-support>

12. llama.cpp / Ollama (LLM runtime)

BLUF ★★★★★ MUST 채택 (LLM 도입 시). GGUF 양자화 + cross-platform LLM runtime의 사실상 표준. Ollama는 llama.cpp 위 wrapper로 OpenAI-compatible API + 모델 자동 다운로드 + RAM/VRAM 자동 관리. BRAVO 17 D7V 8GB GPU에서 Gemma 2 2B / Qwen 2.5 1.5B Q4 모두 30+ tok/s, RPi5 ARM CPU에서도 1B 모델 5-10 tok/s 가능.

1. 개요

항목	내용
llama.cpp 라이선스	MIT
Ollama 라이선스	MIT
관리	ggerganov (llama.cpp), Ollama Inc.
기반 구조	GGUF (양자화 포맷) + Metal/CUDA/Vulkan/CPU
양자화	Q2_K / Q3_K / Q4_K_M / Q5_K_M / Q6_K / Q8_0 / FP16
권장	Q4_K_M (파일 크기 50%, 정확도 거의 동일)
API	OpenAI-compatible REST (Ollama)
모델 카탈로그	https://ollama.com/library

2. 적용 방법

2.1 Ollama 설치

```
# Linux 설치
curl -fsSL https://ollama.com/install.sh | sh

# 또는 Docker
docker run -d --gpus=all -v ollama:/root/.ollama \
  -p 11434:11434 --name ollama ollama/ollama

# 모델 다운로드 (Q4 양자화 자동)
ollama pull gemma2:2b-instruct-q4_K_M      # 1.5 GB
ollama pull qwen2.5:1.5b-instruct-q4_K_M  # 1.0 GB
ollama pull llama3.2:1b-instruct-q4_K_M   # 0.7 GB

# 실행 (CLI test)
ollama run gemma2:2b "안녕하세요"

# OpenAI-compatible API
# http://localhost:11434/v1/chat/completions
```

2.2 모델 크기별 성능 (FALCON-1 측정 예상)

모델	Q4 크기	VRAM	RTX 3060 tok/s	RTX 4060 tok/s	RPi5 CPU tok/s	한국어
llama3.2:1b	0.7 GB	1.2 GB	60 tok/s	100 tok/s	10 tok/s	□ 보통
qwen2.5:1.5b	1.0 GB	1.5 GB	50 tok/s	80 tok/s	7 tok/s	✓ 우수
gemma2:2b	1.5 GB	2.0 GB	40 tok/s	70 tok/s	5 tok/s	✓ 강함
qwen2.5:3b	2.0 GB	2.5 GB	30 tok/s	50 tok/s	✗	✓ 우수
llama3.2:3b	2.0 GB	2.5 GB	30 tok/s	50 tok/s	✗	□ 보통
phi3.5:mini	2.5 GB	3.0 GB	25 tok/s	40 tok/s	✗	□ 보통

3. 하드웨어 필요·제약

항목	최소	권장	FALCON-1 적합성
GPU	RTX 3060 6GB (선택)	RTX 4060+ 8GB	✓TUF + BRAVO 모두 가능
VRAM (Q4 1-2B)	1-2 GB	—	✓
CPU only fallback	ARM A76 4코어	x86 quad+	✓RPi5 1B 모델 가능
RAM	4 GB	8 GB+	✓
디스크	5 GB (모델 캐시)	—	✓
언어	한국어 (모델 의 존)	—	□ 모델별 다름

4. 장점

- 15. MIT 라이선스 상용 친화
- 16. Cross-platform — Linux + macOS + Windows + ARM (RPi5 포함)
- 17. OpenAI-compatible API — instructor·LangChain 등 모든 도구 호환
- 18. 자동 모델 관리 — pull/list/rm CLI
- 19. Q4 양자화 — 파일 크기 50%, 정확도 거의 동일
- 20. GPU 자동 감지 — CUDA/Metal/Vulkan/ROCm 자동 선택
- 21. Multi-GPU 지원 — 큰 모델 분산
- 22. VRAM 자동 관리 — 동시 운용 모델 swap

5. 단점·한계

- 23. Streaming 부분만 — 일부 도구에서 미지원
- 24. Fine-tuning 불가 — Ollama 는 추론만, 학습은 별도 (Unsloth/Axolotl)
- 25. VRAM 사용량 모니터링 — 동시 모델 swap 시 latency spike

- 26. ROS 2 통합 wrapper 약함 — 자체 노드 작성 의무
- 27. Model 다운로드 시간 — 1-2GB 모델 ~5-10 분
- 28. Function calling 지원은 모델별 — 일부 모델 약함 (Llama 3.2 1B 등)

6. 대안 비교

항목	Ollama (llama.cpp)	vLLM	TGI (HuggingFace)	OpenAI API
라이선스	MIT	Apache	Apache	API 비용
RPi5 ARM	✓	✗	✗	—
배포 단순	✓✓	복잡	복잡	API
Multi-GPU	기본	✓✓ 우월	✓	—
throughput	단일 user	고처리량 다중	고처리량	—
FALCON-1	✓MUST	v1.2+ production	비채택	비채택

7. FALCON-1 적용 결정

결정 항목	값
채택 여부	MUST (LLM 도입 시)
도입 시점	Sprint 4 (Integrated Demo)
처리 위치 primary	BRAVO 17 D7V (RTX 4060+ 8GB)
처리 위치 fallback	RPi5 ARM CPU (1B 모델만)
Runtime	Ollama (Docker)
API endpoint	http://localhost:11434/v1
주 모델	I3 Gemma 2 2B Q4 또는 I4 Qwen 2.5 1.5B Q4

결정 항목	값
주요 의존성	I1 Function calling layer, GPU CUDA

8. 참고

- Ollama : <https://ollama.com/>
- llama.cpp : <https://github.com/ggerganov/llama.cpp>
- GGUF 포맷 : <https://huggingface.co/docs/hub/gguf>
- Model 카탈로그 : <https://ollama.com/library>
- Tool support : <https://ollama.com/blog/tool-support>

13. Gemma 2 2B (Google)

BLUF ★★★★★ SHOULD primary. Google Gemma 2 family의 2B instruct 모델, 한국어 강함 (학습 코퍼스 다국어 포함). Q4 1.5GB로 BRAVO 8GB GPU에 YOLOv11-m + Whisper-medium 과 동시 운용 가능. Function calling 지원, instructor 호환. v1.0 PoC LLM primary 후보.

1. 개요

항목	내용
풀네임	Gemma 2 2B Instruct
출시	2024년 6월
라이선스	Gemma Terms of Use (상용 가능, 일부 제한)
관리	Google DeepMind
파라미터	2 B

항목	내용
context length	8192 token
VRAM (Q4_K_M)	1.5 GB
한국어	✔강함 (다국어 학습)
Function calling	✔JSON 출력 안정

2. FALCON-1 사용 시나리오

- v1.0 PoC LLM primary — Sprint 4 도입
- Function calling layer (I1)와 결합 — instructor.from_openai + JSON mode
- BRAVO 8GB 에 YOLOv11-m (5GB) + Whisper-medium (2.5GB) 동시 — Gemma 2 2B (1.5GB)는 한계선
- Rule-based fallback 시점 — '망치 가져와서 박스 빨강에 넣고 드라이버는 녹색' 같은 복합 명령

3. 한국어 정확도 측정 (FALCON-1 권장 목표)

task	목표 정확도
단순 명령 → JSON action (10-15 동사)	≥ 95%
복합 명령 (2-3 step)	≥ 80%
모호 명령 ("그거 가져와")	≥ 50% (Pydantic 검증으로 거부)
Out-of-vocabulary 거부율	≥ 90%

4. 하드웨어 필요·제약

항목	요구	FALCON-1 적합성
GPU	RTX 3060 6GB+	✔TUF, BRAVO
VRAM (Q4)	1.5 GB	✔BRAVO 한계선 (다른 모델과 동시)
CPU fallback	RPi5 ARM에서 5 tok/s	□ marginal, 1B 모델 권장
latency	200-500ms (action 생성)	✔
max tokens (action)	100-200 token (JSON)	✔

5. 장점·단점

장점 :

- Google 검증 모델 — production-ready
- 한국어 강함 — Llama/Phi 대비 우월
- Function calling 안정 — JSON 출력 신뢰성
- Q4 1.5GB 로 작음
- Gemma Terms — 상용 가능

단점 :

- Gemma Terms — 일부 use case 제한 (military 등)
- Context length 8K — 짧음 (긴 history 는 truncate)
- Llama·Qwen 대비 커뮤니티 작음

6. FALCON-1 적용 결정

결정 항목	값
채택	SHOULD primary (Sprint 4)
도입 시점	Sprint 4 (Integrated Demo)

결정 항목	값
처리 위치	BRAVO 17 D7V (RTX 4060+ 8GB)
양자화	Q4_K_M (1.5 GB)
대안 (한국어 더 강함 시)	I4 Qwen 2.5 1.5B (Sprint 4 비교)
주요 의존성	I1 Function calling, I2 Ollama

7. 참고

- Gemma 2 : https://ai.google.dev/gemma/docs/get_started
- HuggingFace : <https://huggingface.co/google/gemma-2-2b-it>
- Ollama : <https://ollama.com/library/gemma2>

I4. Qwen 2.5 1.5B/3B (Alibaba)

BLUF ★★★★★ SHOULD 대안 (Gemma 2와 양자택일). Alibaba 2024-09 공개, Gemma 2 대비 한국어 약간 우수 + Apache 2.0 라이선스 (상용 자유). 1.5B Q4 1.0GB로 매우 가벼움, 3B는 정확도 +5% trade-off. Sprint 4 Gemma 2와 비교 시험 후 결정.

1. 개요

항목	내용
풀네임	Qwen 2.5 1.5B / 3B Instruct
출시	2024년 9월
라이선스	Apache 2.0
관리	Alibaba (Tongyi Qianwen 팀)

항목	내용
파라미터	1.5 B / 3 B / 7 B / 14 B / ...
context length	32K (1.5B/3B)
VRAM (Q4)	1.0 GB (1.5B), 2.0 GB (3B)
한국어	✔우수 (CJK 포함 다국어)
Function calling	✔안정

2. Gemma 2 vs Qwen 2.5 비교 (FALCON-1)

항목	Gemma 2 2B (I3)	Qwen 2.5 1.5B (I4)
라이선스	Gemma Terms	Apache 2.0 (자유)
VRAM Q4	1.5 GB	1.0 GB (50% 작음)
context length	8K	32K (4x)
한국어	✔강함	✔우수 (약간 +)
Function calling	✔	✔
커뮤니티	Google 공식	Alibaba + 활발한 fine-tune
FALCON-1	primary 후보	대안 (또는 primary)

3. FALCON-1 적용 결정

결정 항목	값
채택	SHOULD 대안 (Gemma 2와 양자택일)
Sprint 4 비교	Gemma 2 vs Qwen 2.5 한국어 정확도 측정
Apache 2.0 우월 시	Qwen 2.5 primary 채택 (상용 자유)

결정 항목	값
양자화	Q4_K_M
context length 가치	32K로 긴 history 보존 (v1.1+)

4. 참고

- HuggingFace : <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct>
- Ollama : <https://ollama.com/library/qwen2.5>
- Tech report : <https://qwenlm.github.io/blog/qwen2.5/>

15. Llama 3.2 1B/3B (Meta)

BLUF ★★★ 대안 (RPi5 fallback). Meta 2024-09 출시 1B/3B small models, 0.7-2.0 GB Q4로 가장 작음. 한국어 정확도 Gemma-Qwen 대비 약간 약함. RPi5 ARM CPU에서 1B 10 tok/s 가능 — fallback 시나리오에서 의미. v1.0 primary는 Gemma 또는 Qwen, RPi5 단독 운용 시 Llama 3.2 1B 검토.

1. 개요 + 적용 결정

항목	내용
출시	2024년 9월
라이선스	Llama 3.2 Community License (상용 가능, MAU 700M+ 시 별도 협의)
관리	Meta AI
파라미터	1 B / 3 B (small), 11 B / 90 B (vision)
VRAM Q4	0.7 GB (1B), 2.0 GB (3B)
한국어	□ 보통 (영어 대비 약함)

항목	내용
RPi5 1B 속도	10 tok/s

2. FALCON-1 적용 결정

결정 항목	값
v1.0 primary	비채택 (Gemma·Qwen 한국어 우월)
RPi5 fallback	Llama 3.2 1B Q4 — BRAVO GPU 부재 시
참고 가치	Meta 검증 모델, 학회 baseline

3. 참고

- HuggingFace : <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct>
- Ollama : <https://ollama.com/library/llama3.2>

I6. Phi-3.5 mini (Microsoft)

BLUF ★★★ 대안. Microsoft 2024-08 공개 3.8B mini 모델, MIT 라이선스. 영어 reasoning 강함, 한국어 정확도 Gemma·Qwen 대비 약함. 2.5 GB Q4로 BRAVO에서 운용 가능하나 한국어 작업에는 1순위 아님. 학회 baseline 비교용.

1. 개요 + 적용 결정

항목	내용
풀네임	Phi-3.5 mini Instruct
출시	2024년 8월

항목	내용
라이선스	MIT
파라미터	3.8 B
context length	128K (긴 context)
VRAM Q4	2.5 GB
한국어	□ 보통
영어 reasoning	✔강함

2. FALCON-1 적용 결정

결정 항목	값
v1.0 채택	비채택 (한국어 약함)
참고 가치	영어 시연 시 (학회·해외 발표) 검토
MIT 라이선스 강점	v1.2+ 상용 시 가치

3. 참고

- HuggingFace : <https://huggingface.co/microsoft/Phi-3.5-mini-instruct>
- Ollama : <https://ollama.com/library/phi3.5>

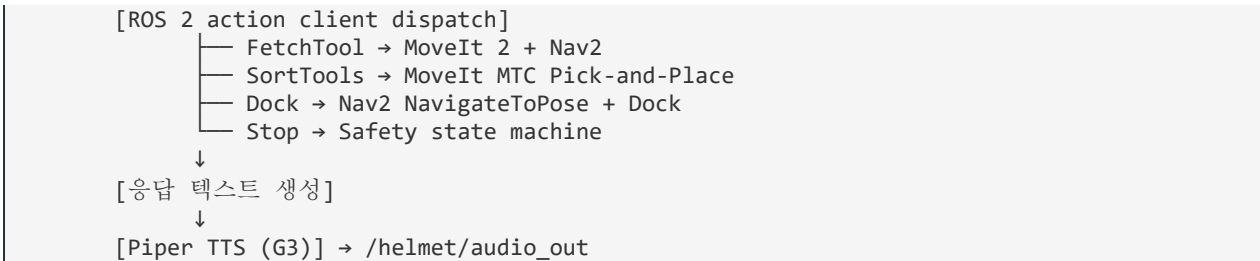
영역 I 종합 결론

1. 채택 / 비채택 정리

#	기술	v1.0 채택	도입 Sprint	역할
I1	Function calling layer	✓MUST (LLM 도입 시)	Sprint 4	Pydantic schema 검증
I2	llama.cpp / Ollama	✓MUST (LLM 도입 시)	Sprint 4	LLM runtime
I3	Gemma 2 2B	SHOULD primary	Sprint 4	한국어 강함, BRAVO
I4	Qwen 2.5 1.5B/3B	SHOULD 대안	Sprint 4	Apache 2.0 + 한국어 우수
I5	Llama 3.2 1B/3B	RPi5 fallback	Sprint 4	GPU 부재 시
I6	Phi-3.5 mini	비채택 (한국어 약함)	—	영어 시연 시 검토

2. 통합 LLM Pipeline (FALCON-1 v1.0 Sprint 4+)





3. v1.0 권장 정책 — LLM 도입 vs 비도입 결단

옵션	장점	단점	권장
A. LLM 비도입 (Rule-based 단독)	단순, 빠름 (<5ms), 100% 결정적, 디버깅 쉬움	복합·새 표현 처리 불가, narrative 약함	v1.0 PoC primary 권장
B. Rule-based + LLM fallback	Rule 빠름 + LLM 유연	Sprint 4 시간 +20h, narrative 강화	Sprint 4 시간 여유 시
C. LLM primary (Function calling 의무)	narrative 강력, 자유 명령	Sprint 4 시간 +40h, latency 200-500ms 부담, hallucination risk	v1.1+ 검토

CTO 권장 : 옵션 B — Rule-based 10-15 patterns로 95% 명령 처리 + LLM fallback으로 narrative 강화. Sprint 4 시간 부족 시 옵션 A로 후퇴.

4. Sprint 별 도입 list

Sprint	도입 기술	주요 작업
Sprint 3	Rule-based regex 패턴	10-15 한국어 명령 → action 매핑, Pydantic schema 정의
Sprint 4 (옵션 B)	I1 + I2 + I3 (Gemma) 또는 I4 (Qwen)	Ollama 설치, instructor 통합, fallback 시연
Sprint 5	burn-in 한국어 명령 100회 측정	Rule + LLM 결합 정확도 검증

5. 잔여 위험 및 대응

#	위험	심각도	대응
1	BRAVO VRAM 한계 (Whisper + YOLO + LLM 동시)	▣ Med-High	동적 load/unload, 또는 RPi5 LLM 1B fallback
2	한국어 정확도 95% 미달	▣ Med	Rule-based 보조 비중 ↑, Qwen 2.5 우선
3	Hallucination (잘못된 action 출력)	▣ Med	Pydantic 검증 + retry max=2 + StopAction fallback
4	Sprint 4 시간 부족	▣ Med	옵션 A (Rule-only)로 후퇴 가능
5	LLM latency 200-500ms 영향	▣ Low	Rule-based primary로 95% 명령은 <5ms 처리
6	한국어 학습 데이터 부족	▣ Low	v1.0 어휘 한정으로 충분, fine-tuning 불필요

6. 검증 시험 (Test Plan 매핑)

Test ID	내용	Sprint	성공 기준
T-LLM-01	Rule-based 매핑 정확도 (10 명령)	Sprint 3	≥ 95% (50회)
T-LLM-02	Pydantic 검증 hallucination 거부	Sprint 4	잘못된 JSON 100% 거부
T-LLM-03	LLM fallback 한국어 정확도	Sprint 4	≥ 80% (모호 명령)
T-LLM-04	End-to-end latency (rule)	Sprint 4	< 5ms
T-LLM-05	End-to-end latency (LLM)	Sprint 4	< 700ms
T-LLM-06	동시 운용 VRAM (Whisper + YOLO + LLM)	Sprint 5	OOM 발생 0회

7. 다음 작성 영역

영역 I LLM-Reasoning 6종 분석 완료. 후속 영역 :

- 29. 영역 J — Learning RL · Imitation · VLA (8 종) : Behavior Cloning, ACT, Diffusion Policy, OpenVLA, OpenVLA-OFT+, $\pi_0/\pi_{0.5}$, TinyVLA, GR00T N1.5
- 30. 영역 K — Sim · CI · Logging (8 종)
- 31. 영역 L — Safety · Power (6 종)

부록 — 변경 이력

버전	일자	내용	작성
v0.1	2026-04-30	영역 I LLM-Reasoning 6종 초안 (SLATE BLUE accent)	이건희, CTO 페르소나